Matrices & Graphs — 80/20 Interview Notes

1. The Big Picture

The central idea of this chapter is:

A matrix can represent a graph, and graph structure can reveal useful properties of the matrix.

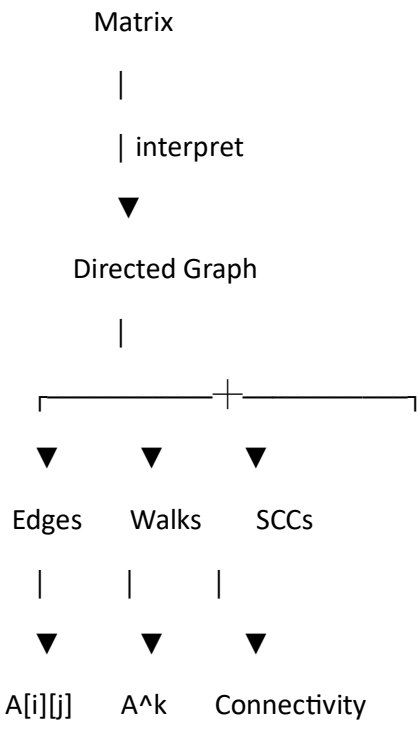
For a nonnegative matrix

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}$$

interpret:

- **row** i→ source node i
- **column** j→ destination node j
- $a_{ij} > 0$→ directed edge $i \to j$
- a_{ij}→ edge weight
- $a_{ij} = 0$→ no edge

Mental model



This is the key conceptual bridge.

---

2. Matrix as a Directed Weighted Graph

Definition

For a nonnegative matrix $A \in \mathbb{R}^{n\times n}$, construct a directed weighted graph:

$$(i,j) \in E \iff a_{ij} > 0$$

and assign

$$w(i,j) = a_{ij}$$

So:

$$\boxed{a_{ij} > 0 \Leftrightarrow i \to j}$$

Example

Consider:

$$A = \begin{bmatrix} 0.5 & 1 & 0 \\ 0.2 & 0 & 2.2 \\ 1.8 & 2 & 0 \end{bmatrix}$$

Translate row by row.

Row 1

$$[0.5, 1, 0]$$

means:

1 —0.5—▶ 1

|

└—1——▶ 2

There is no $1 \to 3$.

Row 2

$$[0.2, 0, 2.2]$$

means:

2 —0.2—▶ 1

|

└—2.2—▶ 3

Row 3

$$[1.8, 2, 0]$$

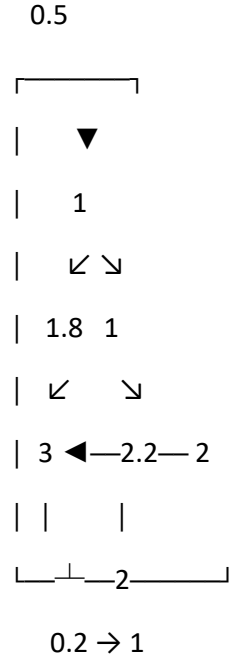
means:

3 —1.8—▶ 1

|

└—2——▶ 2

Full graph:



The exact drawing is less important than the rule:

Matrix entry = directed edge from row node to column node.

---

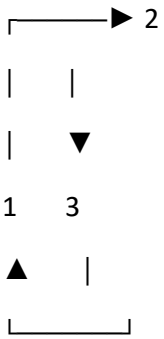
3. Adjacency Matrix

When the weights are only 0or 1, the matrix is commonly called an **adjacency matrix**.

For example:

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

means:



Edges:

$$1 \to 2$$
$$1 \to 3$$
$$2 \to 3$$
$$3 \to 1$$

Interview point

For an $n \times n$adjacency matrix:

- space = $O(n^2)$
- checking whether edge $i \to j$exists = $O(1)$
- iterating over all neighbors of a node = $O(n)$

Compare with adjacency lists:

Representation	**Space**	**Check edge**	**Iterate neighbors**
Matrix	$O(V^2)$	$O(1)$	$O(V)$
Adjacency list	$O(V+E)$ usually $O\big(\deg(v)\big)$		$O\big(\deg(v)\big)$

For sparse graphs, adjacency lists are generally much more memory-efficient.

---

4. Matrix Multiplication = Combining Paths

This is one of the **most important concepts in the chapter**.

Suppose:

$$A^2 = A \cdot A$$

Then

$$(A^2)_{ij} = \sum_{k=1}^{n} a_{ik}\, a_{kj}$$

Why does this represent two-step walks?

Because:

$$i \to k \to j$$

has weight:

$$a_{ik}a_{kj}$$

and we sum over every possible intermediate node k.

Therefore:

$$\boxed{(A^2)_{ij} = \text{total weight of all 2-step walks from } i \text{ to } j}$$

For an unweighted adjacency matrix, it becomes:

$$\boxed{(A^2)_{ij} = \text{number of 2-step walks from } i \text{ to } j}$$

---

5. Example: A^2 Counts Two-Step Walks

Consider:

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Calculate:

$$A^2 = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

Look at:

$$(A^2)_{11} = 1$$

There is one two-step walk from node 1 back to node 1:

1 → 3 → 1

Look at:

$$(A^2)_{12} = 0$$

There is no two-step walk:

1 → ? → 2

Look at:

$$(A^2)_{13} = 1$$

There is one:

1 → 2 → 3

---

6. The General Result: A^k

This is a **high-value interview concept**.

For an unweighted graph:

$$\boxed{(A^k)_{ij} = \text{number of walks of length } k \text{ from } i \text{ to } j}$$

So:

$$A$$

→ walks of length 1

$$A^2$$

→ walks of length 2

$$A^3$$

→ walks of length 3

and generally:

$$A^k$$

→ walks of length k.

Visual

A

|

└──▶ 1-step walks

A^2

|

└──▶ 2-step walks

A^3

|

└──▶ 3-step walks

A^k

|

└──▶ k-step walks

Important distinction

A **walk** may revisit nodes.

Don't confuse:

- walk
- path

- simple path
- cycle

For A^k, the relevant concept is **walk**, not necessarily a simple path.

---

7. Weighted Graph Interpretation

For weighted graphs:

$$(A^k)_{ij}$$

is not simply the number of walks.

Instead, it is the **sum of products of edge weights over all length-k walks**.

Suppose:

1 --2--> 2 --3--> 3

The weight of:

$$1 \to 2 \to 3$$

is:

$$2 \times 3 = 6$$

So the matrix multiplication naturally performs:

multiply along a walk

and

sum across possible walks

This idea becomes surprisingly important in advanced graph algorithms and ML.

---

8. Walk

Definition

A directed walk is a sequence:

$$v_1, v_2, \ldots, v_k$$

such that every consecutive pair has an edge:

$$(v_i, v_{i+1}) \in E$$

Example:

1 → 2 → 3 → 1 → 2

is a valid walk if those edges exist.

A node can appear multiple times.

Interview question

Q: Can a walk contain the same node multiple times?

Yes.

That's one of the differences between a walk and a simple path.

---

9. Reachability

A node $v$is **reachable** from $u$if there exists a directed walk:

$$u \rightsquigarrow v$$

Example:

A → B → C

Then:

A can reach B

A can reach C

B can reach C

But:

C cannot reach A

unless another edge exists.

---

10. Strong Connectivity

This is another **P0 interview concept**.

Definition

A directed graph is **strongly connected** if:

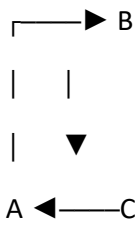
$$\forall u, v \in V:$$

there is a directed walk from $u$to v.

In simple language:

Every node can reach every other node.

Example:



If:

A → B

B → C

C → A

then:

A → B → C → A

and every node can reach every other node.

Therefore the graph is strongly connected.

---

11. Connected vs Strongly Connected

Very common interview trap.

Undirected graph

A —— B —— C

There is a connection between all nodes.

Directed graph

A ——▶ B ——▶ C

All nodes are reachable **in one direction**, but:

C –X→ A

Therefore it is **not strongly connected**.

Remember

Strong connectivity requires two-way reachability through directed paths.

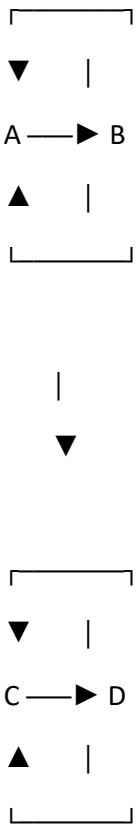
It does NOT mean every pair must have direct edges.

---

12. Strongly Connected Components (SCC)

An SCC is a maximal set of nodes such that every node can reach every other node in that set.

Example:



Potential SCCs:

SCC1 = {A, B}

SCC2 = {C, D}

If there is:

B → C

then:

SCC1 → SCC2

but if no route returns from SCC2 to SCC1, they remain separate SCCs.

---

13. SCCs and Matrices

This is where the chapter's main mathematical idea becomes useful.

If:

$$A$$

is a nonnegative matrix, construct its directed graph.

Then:

Graph SCC structure

↓

Matrix block structure

If the matrix is irreducible:

$$\boxed{\text{digraph strongly connected}}$$

If it isn't:

$$\boxed{\text{matrix is reducible}}$$

---

14. Irreducible Matrix

Definition

For a nonnegative matrix A:

$$\boxed{A \text{ is irreducible} \iff G(A) \text{ is strongly connected}}$$

Otherwise:

$$\boxed{A \text{ is reducible}}$$

Example

Consider:

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Graph:

1 → 2 → 3

↑ |

└──────┘

Every node reaches every other node.

Therefore:

$$\boxed{A \text{ is irreducible}}$$

---

15. Reducible Matrix

Consider:

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

Graph:

1 → 2 → 3

There is no way to get from 3 back to 1.

Therefore the graph isn't strongly connected.

Hence:

$$\boxed{A \text{ is reducible}}$$

---

16. Why SCCs Matter

This is more useful than the terminology itself.

Suppose a huge graph contains:

10,000 nodes

but they form:

SCC A → SCC B → SCC C

You can reason about the system at the component level:

SCC A

|

▼

SCC B

|

▼

SCC C

Instead of reasoning about thousands of individual nodes.

This is called **condensing** the graph.

The resulting SCC graph is a **DAG** (directed acyclic graph).

Important interview fact

The condensation graph of a directed graph, obtained by collapsing each SCC into one node, is always a DAG.

---

17. Frobenius Normal Form

This is the more mathematical portion.

After rearranging the nodes appropriately, a reducible nonnegative matrix can be written in block upper-triangular form:

$$P^T AP = \begin{bmatrix} A_1 & A_{12} & \cdots & A_{1k} \\ 0 & A_2 & \cdots & A_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_k \end{bmatrix}$$

where each diagonal block:

$$A_i$$

corresponds to an SCC.

Mental model

Original matrix

|

▼

Construct graph

|

▼

Find SCCs

|

▼

Reorder nodes

|

▼

Block upper-triangular matrix

---

18. Why Is There a Permutation Matrix?

A permutation matrix simply rearranges rows/columns.

For example:

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

swaps:

row 1 ↔ row 2

when multiplying from the left.

And:

column 1 ↔ column 2

when multiplying from the right.

Therefore:

$$P^T AP$$

reorders both rows and columns consistently.

Graph interpretation

This is simply:

Renaming/reordering the nodes.

The underlying graph connectivity does not fundamentally change.

---

19. Extremely Important: $P^T AP$

You should understand this rather than memorize it.

If:

$$P$$

is a permutation matrix, then:

$$P^{-1} = P^T$$

Therefore:

$$P^T AP = P^{-1} AP$$

This is a **similarity transformation**.

It changes the representation/order of the matrix while preserving important structural properties such as eigenvalues.

For this chapter, the most important interpretation is:

$$\boxed{P^T AP = \text{same graph structure, different node labeling}}$$

---

20. Frobenius Normal Form — What to Actually Know for Interviews

P0

Know:

- SCCs
- reducible vs irreducible

- permutation matrices
- block upper-triangular structure
- graph ↔ matrix correspondence

P1

Understand:

- Frobenius normal form
- why SCC ordering produces the block structure
- condensation DAG

P2

Usually don't spend significant time memorizing:

- formal proof of the Frobenius theorem
- detailed component-class numbering procedure
- lengthy proof constructions

For a typical ML/software engineering interview, these are much lower ROI.

---

21. Implementation — Build a Matrix → Graph

For AI/ML engineering, Python is the natural implementation language.

```
def matrix_to_graph(A):
    n = len(A)
    graph = [[] for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if A[i][j] > 0:
                graph[i].append((j, A[i][j]))

    return graph
```

Example:

```
A = [
    [0, 1, 1],
    [0, 0, 1],
    [1, 0, 0]
]

graph = matrix_to_graph(A)

print(graph)
```

Output conceptually:

```
[
    [(1, 1), (2, 1)],
    [(2, 1)],
    [(0, 1)]
]
```

Complexity:

$$O(n^2)$$

because every matrix entry is inspected.

---

22. Graph → Adjacency Matrix

The reverse operation:

```
def graph_to_matrix(graph):
    n = len(graph)
    A = [[0] * n for _ in range(n)]

    for u in range(n):
        for v, weight in graph[u]:
```

```
            A[u][v] = weight

    return A
```

This demonstrates the chapter's core idea:

Matrix

⇕

Graph

They are two representations of related information.

---

23. SCC Implementation — Kosaraju

You should know **what SCC algorithms do**, even if you're not expected to implement one from memory in an ML interview.

Kosaraju's algorithm:

1. DFS on original graph.
2. Store nodes by finishing time.
3. Reverse all edges.
4. DFS in reverse finishing order.
5. Each DFS tree = one SCC.

Complexity:

$$\boxed{O(V+E)}$$

Implementation:

```
def kosaraju(graph):
    n = len(graph)
    visited = [False] * n
    order = []

    def dfs1(u):
```

```
    visited[u] = True

    for v in graph[u]:
        if not visited[v]:
            dfs1(v)

    order.append(u)

for u in range(n):
    if not visited[u]:
        dfs1(u)

# Reverse graph
rev = [[] for _ in range(n)]

for u in range(n):
    for v in graph[u]:
        rev[v].append(u)

visited = [False] * n
components = []

def dfs2(u, component):
    visited[u] = True
    component.append(u)

    for v in rev[u]:
        if not visited[v]:
```

```
            dfs2(v, component)

    for u in reversed(order):
        if not visited[u]:
            component = []
            dfs2(u, component)
            components.append(component)

    return components
```

Example:

```
graph = [
    [1],
    [2],
    [0, 3],
    [4],
    [3]
]

print(kosaraju(graph))
```

Conceptually:

[[0, 1, 2], [3, 4]]

because:

0 → 1 → 2 → 0

forms one SCC, while:

3 ↔ 4

forms another.

---

24. Matrix Powers in Python

For small matrices, NumPy makes the idea obvious:

```
import numpy as np


A = np.array([
   [0, 1, 1],
   [0, 0, 1],
   [1, 0, 0]
])


A2 = A @ A
A3 = A @ A @ A


print(A2)
print(A3)
```

Here:

A @ A

is ordinary matrix multiplication.

The important conceptual point:

A → 1-step walks

A @ A → 2-step walks

A @ A @ A → 3-step walks

---

25. Interview Example: Social Network

Imagine:

A = adjacency matrix of users

where:

$$A_{ij} = 1$$

means:

user $i$follows user j.

Then:

$$(A^2)_{ij}$$

tells you how many two-step follow relationships connect $i$to j.

Example:

Alice → Bob

Bob → Charlie

Then:

Alice → Bob → Charlie

is a 2-hop connection.

Therefore:

$$(A^2)_{\text{Alice,Charlie}} > 0$$

This is a very practical interpretation of the mathematics.

---

26. Interview Example: Web Graph

Think of:

page → page

as a directed graph.

Then:

$$A_{ij} = 1$$

means page $i$links to page j.

Now:

$$A^2$$

captures two-link navigation.

A^3

captures three-link navigation.

This kind of graph representation is foundational to many graph-based algorithms.

---

27. Interview Example: Recommendation Systems

Suppose:

User → Product

is represented by a bipartite graph.

You can reason about:

User

↓

Product

↓

Other User

or:

User

↓

Product

↓

Category

as graph walks.

This is one reason graph representations can become useful in recommendation systems and graph ML.

---

28. Interview Example: Dependency Graph

Consider:

Service A → Service B → Database

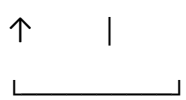
or:

Task A → Task B → Task C

A directed graph naturally represents dependencies.

SCCs become especially interesting if you discover:

Service A → Service B



Now A and B form an SCC.

That can indicate a **cyclic dependency**.

For software engineering interviews, this is much more practically relevant than memorizing Frobenius normal-form terminology.

---

29. Matrix Representation vs Graph Representation

Interviewers may ask:

"Why would I use a graph if I already have a matrix?"

Because each representation makes different operations convenient.

Matrix

Good for:

- $O(1)$edge lookup
- matrix algebra
- matrix powers
- linear algebra
- dense graphs

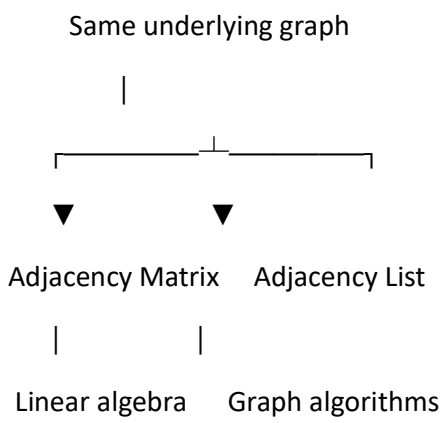
Graph adjacency list

Good for:

- sparse graphs
- BFS/DFS
- neighbor traversal
- SCC algorithms

- graph algorithms

Mental model



---

30. A Very Important ML Connection

The chapter is mathematically introducing an idea that appears later in **Graph Machine Learning**.

Suppose node features are stored in:

$$X \in \mathbb{R}^{n \times d}$$

and the graph is represented by an adjacency matrix:

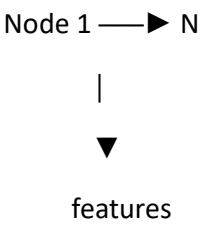
$$A \in \mathbb{R}^{n \times n}$$

Then operations such as:

$$AX$$

combine information from neighboring nodes.

For example:



The adjacency matrix determines **which nodes exchange information**.

This is foundational intuition behind graph neural networks.

You don't need to study GNNs here yet, but this chapter provides an important prerequisite.

---

31. One Important Correction to Keep in Mind

The chapter focuses on **nonnegative matrices**.

That matters because the graph is constructed using:

$$a_{ij} > 0$$

as the presence of an edge.

For arbitrary matrices containing negative values, interpreting every nonzero entry as a positive weighted graph edge requires a slightly different convention.

For your interview preparation, remember:

The irreducible/reducible definitions in this chapter are specifically being applied to nonnegative square matrices.

---

32. What You Should Memorize

Don't memorize the entire chapter.

Memorize these relationships:

1. Matrix → Graph

$$\boxed{a_{ij} > 0 \Rightarrow i \to j}$$

2. Matrix square

$$\boxed{(A^2)_{ij} = \sum_k a_{ik}\, a_{kj}}$$

3. Unweighted A^k

$$\boxed{(A^k)_{ij} = \#\text{ walks of length } k}$$

4. Strong connectivity

$$\boxed{\text{every node can reach every other node}}$$

5. SCC

$$\boxed{\text{maximal mutually reachable set}}$$

6. Irreducibility

$$\boxed{A \text{ irreducible } \iff G(A) \text{ strongly connected}}$$

7. Permutation

$$\boxed{P^T AP = \text{reordering/relabeling}}$$

8. Frobenius normal form

$$\boxed{P^T AP = \text{block upper-triangular matrix}}$$

with SCC-related blocks on the diagonal.

---

33. Interview Questions You Should Be Able to Answer

P0 — Must Know

Q1. How can a matrix represent a directed graph?

Answer:

For a nonnegative matrix A, row $i$represents source node i, column $j$represents destination j, and $a_{ij} > 0$represents an edge $i \to j$with weight a_{ij}.

Q2. What does A^2represent?

For an unweighted adjacency matrix:

$$(A^2)_{ij}$$

is the number of two-step walks from $i$to j.

Q3. What does A^k represent?

$$(A^k)_{ij}$$

counts the number of length-k walks from i to j for an unweighted adjacency matrix.

Q4. What is strong connectivity?

Every node can reach every other node through directed walks.

Q5. What is an SCC?

A maximal set of vertices where every pair of vertices is mutually reachable.

Q6. What is the relationship between SCC and irreducibility?

For a nonnegative square matrix:

$$A \text{ is irreducible}$$

iff its directed graph is strongly connected.

Q7. Why use permutation matrices?

To reorder rows and columns consistently.

In graph terms, this corresponds to **relabeling/reordering nodes**.

Q8. What is $P^T AP$?

A similarity transformation corresponding, for a permutation matrix, to a simultaneous row/column permutation.

In this context:

Same graph structure, different node ordering.

---

34. P1 Interview Questions

Q9. Why is matrix multiplication related to graph walks?

Because:

$$(A^2)_{ij} = \sum_k A_{ik} A_{kj}$$

considers every possible intermediate node k.

Q10. What is the difference between connected and strongly connected?

For directed graphs, strong connectivity requires directed reachability in both directions between every pair.

Q11. What happens when SCCs are collapsed?

You obtain the **condensation graph**, which is a DAG.

Q12. Why does Frobenius normal form have zeros below the diagonal?

After ordering SCCs according to the condensation DAG, edges only go from earlier components toward later components.

Hence the corresponding matrix becomes block upper triangular.

---

35. P2 — Low ROI for Your Current Goal

Don't spend much time on:

- reproducing the complete Frobenius normal-form proof
- memorizing the exact component-class numbering construction
- formal set-theoretic proof of SCC equivalence relations
- memorizing every matrix displayed in the chapter

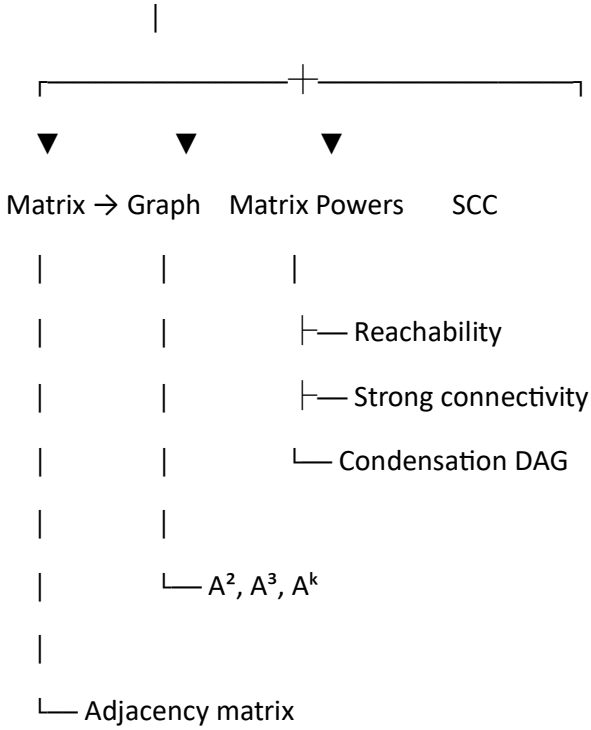
Understand the **idea**, not the proof machinery.

---

36. 80/20 Study Map

For your interview preparation, I would allocate the chapter approximately like this:

MATRICES + GRAPHS



↓

Irreducibility

↓

Frobenius Normal Form

↓

Permutation Matrix

Your priority

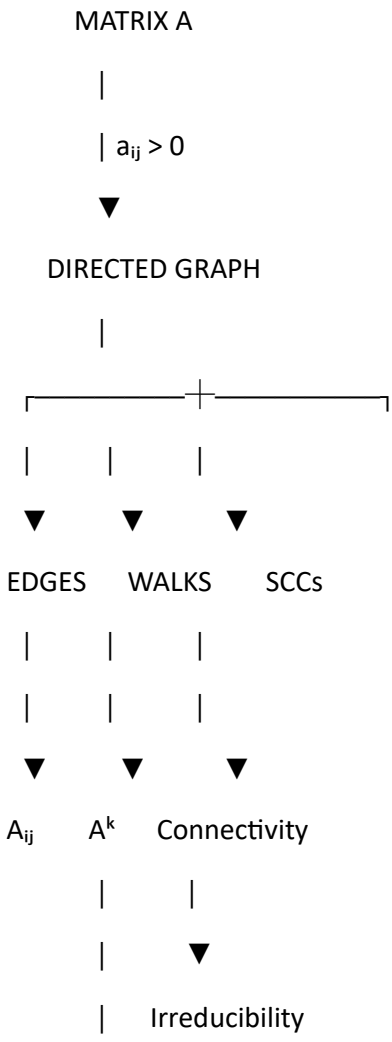
Concept	**Priority**	**Interview value**
Matrix ↔ graph	**P0**	Very high
Adjacency matrix	**P0**	Very high
A^k ↔ walks	**P0**	High
Walk/reachability	**P0**	Very high
Strong connectivity	**P0**	Very high

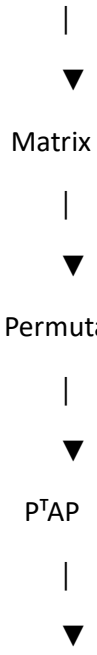
Concept	Priority	Interview value
SCC	**P0**	Very high
SCC algorithms	**P0/P1**	Very high
Reducible/irreducible	**P1**	Moderate
Permutation matrix	**P1**	Moderate
Frobenius normal form	**P1/P2**	Lower
Formal proof	**P2**	Low

---

37. Final Mental Model

If you remember only one diagram from this chapter, remember this:





The 5 things I'd expect you to be able to explain without notes

1. **How an adjacency matrix represents a graph.**
2. **Why A^2counts 2-step walks and A^kcounts k-step walks.**
3. **What strong connectivity and SCC mean.**
4. **Why SCC structure corresponds to block structure in a matrix.**
5. **Why $P^T AP$represents reordering/relabeling of nodes.**

If you can explain those five with a small example and implement **matrix→graph + SCC**, you've captured most of the interview-relevant value of this chapter.